



**RAJALAKSHMI ENGINEERING COLLEGE**

*Approved by AICTE | Affiliated to Anna University | Accredited by NAAC*

# **EVENTURA AI**

**An Agentic AI Event Planning and Coordination Platform**

## **AD23731 PROJECT REPORT**

**Agentic AI**

*Submitted by*

**Amirtha Raja Rajeswari N A**

**Year/Sem : IV, VIII**

**2116231501012**

**Department of Artificial Intelligence and Machine Learning  
Rajalakshmi Engineering College**

**Faculty Guide: Dr. M. Soundarya**

**Academic Year: 2026 - 2027**

## ABSTRACT

---

Event planning is a complex decision-making problem involving multiple constraints such as budget, location, guest capacity, vendor availability, service requirements, preferences, and scheduling. Conventional event-planning applications generally provide search, filtering, recommendation, or booking functionality, leaving users responsible for coordinating the overall planning process. Although large language models can generate plans from natural-language instructions, a standalone model may produce unsupported recommendations, overlook hard constraints, exceed the available budget, or lack mechanisms for validation and human approval.

Eventura AI addresses these limitations through an agentic artificial intelligence architecture for event planning and coordination. The system accepts natural-language event requirements and transforms them into structured planning requirements. A set of specialized agents then performs feasibility analysis, planning, research, budget reasoning, negotiation, validation, and booking through a controlled workflow. The system integrates Agentic Retrieval-Augmented Generation (Agentic RAG) with SQL-based vendor retrieval and vector-based knowledge retrieval; retrieval can be refined when the available evidence is insufficient for the planning task.

A Critic Agent provides plan validation and a reflection loop that allows problematic plans to return to the planning stage. A Human-in-the-Loop (HITL) mechanism is positioned before booking, allowing users to approve, modify, or reject proposed decisions. The current implementation uses a synthetic vendor marketplace and a mock booking environment, making it suitable for controlled experimentation without relying on real vendor or customer data.

The system is implemented using React for the frontend, FastAPI and Python for the backend, LangGraph for workflow orchestration, Ollama with Llama 3.1 8B for local language-model inference, and PostgreSQL with pgvector for structured and vector retrieval. The project is designed to compare No-RAG, Basic-RAG, and Agentic-RAG configurations using measurable metrics such as constraint satisfaction, budget compliance, evidence sufficiency, retrieval relevance, conflict detection, replanning success, latency, and retrieval rounds.

**Keywords:** Agentic AI, Multi-Agent Systems, Agentic RAG, LangGraph, Human-in-the-Loop, Event Planning, Constraint-Aware Planning, pgvector.

## TABLE OF CONTENTS

---

|                                                  |    |
|--------------------------------------------------|----|
| 1. INTRODUCTION .....                            | 4  |
| 2. PROBLEM STATEMENT AND BUSINESS CASE .....     | 5  |
| 3. USER AND STAKEHOLDER ANALYSIS .....           | 6  |
| 4. PROPOSED AGENTIC AI SOLUTION .....            | 7  |
| 5. DATASET / KNOWLEDGE SOURCES .....             | 8  |
| 6. TOOLS AND TECHNOLOGIES USED .....             | 9  |
| 7. AGENT ARCHITECTURE .....                      | 9  |
| 8. AGENT WORKFLOW AND ORCHESTRATION .....        | 11 |
| 9. AGENTIC RAG IMPLEMENTATION .....              | 13 |
| 10. SYSTEM IMPLEMENTATION .....                  | 14 |
| 11. EXPERIMENTAL EVALUATION AND RESULTS .....    | 16 |
| 12. COMPARISON, INNOVATION AND SCALABILITY ..... | 17 |
| 13. CONCLUSION AND FUTURE WORK .....             | 19 |
| REFERENCES / PROJECT SOURCES .....               | 20 |

# 1. INTRODUCTION

---

## 1.1 Background and Motivation

Event planning is a multi-stage decision-making activity that requires coordination between several independent requirements. Depending on the type and scale of an event, the organizer may need to identify suitable venues, catering services, decorators, photographers, entertainment providers, and transportation. Each selection is affected by constraints such as budget, location, capacity, date, availability, service category, and user preferences.

Traditional event-planning applications generally operate as search, CRUD, recommendation, or booking systems. Although they help users discover individual services, the responsibility for combining those services into a coherent plan remains with the user, who must compare alternatives, verify constraints, calculate costs, identify conflicts, and modify the plan when a selected service becomes unavailable.

Eventura AI was developed to address this coordination problem through an agentic workflow. It combines specialized agents, retrieval-augmented generation, tool use, structured state, validation, iterative critique, and human approval, and supports weddings, birthdays, and college fests. The motivation is not to build a conversational event-planning chatbot, but a system capable of reasoning over multiple constraints, retrieving supporting evidence, coordinating specialized agents, validating decisions, and revising plans when problems are detected.

There is also a gap between conventional recommendation systems and LLM-based systems. A conventional application may help a user find vendors but does not reason over the full combination of budget, capacity, location, availability, required categories, and evidence. A general-purpose LLM can produce plausible plans but may fabricate vendor details or prices and recommend actions without a controlled approval mechanism. Eventura therefore combines structured data retrieval with generative reasoning and workflow orchestration.

## 1.2 Need for Agentic AI

A conventional chatbot follows a simple interaction model (User Input, Language Model, Response). Event planning, however, requires a sequence of interconnected decisions. Consider the request: “Plan a wedding in Chennai for 200 guests with a budget of ₹5,00,000. I need a venue, catering, decoration, and photography.” The system must determine the event type, location and date, guest count, mandatory categories, vendors that satisfy capacity and availability, the budget distribution, whether the plan stays within budget, whether sufficient evidence supports each recommendation, and whether a rejected or unavailable vendor must be replaced.

Selecting a venue, for example, affects guest capacity, location, the remaining budget, catering requirements, and subsequent vendor selection. A change in one decision may invalidate previous ones. These activities are better represented as a workflow than as a single text-generation call. Eventura therefore uses specialized agents for intake, feasibility, planning, research, budgeting, negotiation, criticism, and booking. The agentic approach provides:

- **Task decomposition:** complex planning is divided into specialized subtasks.
- **Tool use:** agents access structured databases and retrieval systems.
- **State preservation:** information generated in one stage is available to later stages.
- **Validation and reflection:** plans are evaluated and failed plans return to planning.
- **Human intervention:** users approve, modify, or reject decisions.
- **Action execution:** approved plans proceed to controlled mock booking.

### 1.3 Project Objectives

The primary objective is to develop an agentic event-planning system that converts natural-language requirements into validated, actionable plans. The specific objectives are:

1. To convert natural-language event requests into structured requirements.
2. To determine whether an event request is feasible under the specified constraints.
3. To retrieve relevant vendor and domain-knowledge evidence.
4. To select vendors according to event requirements and constraints.
5. To reason about budget allocation and expenditure.
6. To identify conflicts and constraint violations.
7. To critique generated plans and perform corrective iterations.
8. To maintain structured state throughout the planning workflow.
9. To request human approval before booking.
10. To execute controlled mock bookings after approval.
11. To expose agent activity to improve transparency.
12. To compare No-RAG, Basic-RAG, and Agentic-RAG approaches.
13. To evaluate planning quality using measurable metrics.

## 2. PROBLEM STATEMENT AND BUSINESS CASE

---

### 2.1 Problem Definition

The central problem addressed by Eventura AI is: how can an AI system autonomously coordinate multiple event-planning decisions while satisfying user requirements, budget constraints, vendor constraints, and evidence requirements, while still allowing human control before irreversible actions?

Existing workflows require users to perform several disconnected activities: searching for vendors, comparing prices, checking capacity and availability, allocating budgets, coordinating multiple services, resolving conflicts, replanning when requirements change, and confirming final decisions. This creates significant cognitive and operational overhead.

A standalone LLM does not inherently guarantee factual grounding, constraint satisfaction, or controlled action execution; generated recommendations may not be grounded in available data and may violate hard constraints. Eventura addresses this by combining multi-agent orchestration, structured state, Agentic RAG, SQL and vector retrieval, feasibility checking, budget reasoning, negotiation, critic-based validation, replanning, human approval, and controlled booking.

### 2.2 Business and Real-World Context

The event-planning domain has several characteristics that make it suitable for an agentic system:

| Characteristic              | Description                                                                                                |
|-----------------------------|------------------------------------------------------------------------------------------------------------|
| Multi-constraint decisions  | A suitable vendor must satisfy several constraints simultaneously rather than simply having a high rating. |
| Multiple service categories | An event may need venue, catering, decoration, photography, entertainment, transportation, and more.       |
| Budget dependencies         | Selecting one vendor changes the budget available for other categories.                                    |
| Availability dependencies   | A vendor may satisfy every other constraint but be unavailable on the required date.                       |
| Dynamic replanning          | A vendor becoming unavailable or a budget change may invalidate part of an existing plan.                  |
| Need for human control      | Booking and other consequential actions should not rest solely on an autonomous LLM decision.              |

These challenges occur across weddings, birthdays, and college fests, where several vendor decisions must be coordinated rather than handled as independent searches. Eventura AI addresses this through coordinated planning, retrieval, validation, and human approval.

For development and evaluation, the system uses a synthetic vendor marketplace. This provides a controlled environment for testing the agentic workflow without relying on real customer or vendor data; the synthetic vendor information does not represent actual market data.

### 3. USER AND STAKEHOLDER ANALYSIS

---

#### 3.1 Target Users

| User Group                              | Description / Requirement                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------------------------|
| Individual event organizers             | Plan weddings, birthdays, family functions and social events using natural language.        |
| Families                                | Coordinate weddings and family celebrations with shared preferences and approvals.          |
| College clubs and student organizations | Plan fests, technical and cultural events, workshops and competitions within tight budgets. |
| Professional organizers / coordinators  | Use the system for preliminary planning, vendor discovery, budget analysis, and monitoring. |

#### 3.2 Stakeholder Identification

| Stakeholder          | Role                       | Major Requirements               |
|----------------------|----------------------------|----------------------------------|
| Event Organizer      | Primary decision-maker     | Planning, budget, vendors        |
| Family / Client      | Event participant          | Preferences and approvals        |
| College Club         | Event organizer            | Budget and service coordination  |
| Vendor               | Service provider           | Service requirements and booking |
| Event Manager        | Operational coordinator    | Planning and monitoring          |
| System Administrator | System maintenance         | Database and system reliability  |
| AI System / Operator | Decision-support component | Retrieval, planning, validation  |

#### 3.3 User Requirements

##### Functional requirements :

The system should allow the user to: create an event using natural language; specify date, location, guest count, budget and required services; view recommended vendors and budget allocation; inspect retrieved evidence; review agent activity; approve or reject proposed plans; simulate disruptions; and proceed with controlled booking.

##### Non-functional requirements :

The system should provide transparency, traceability, constraint awareness, reliability, responsiveness, controlled autonomy, modularity, extensibility, state persistence, explainability, reproducibility, and maintainability.

## 4. PROPOSED AGENTIC AI SOLUTION

---

### 4.1 Overall Solution

Eventura AI transforms a natural-language event request into a structured, validated, and actionable plan. The system is not based on one LLM call; its workflow combines specialized agents, retrieval, structured state, conditional routing, iterative reasoning, human intervention, and action execution.

User Goal -> Natural-Language Intake -> Requirement Extraction -> Feasibility Analysis

-> Planning -> Research / Agentic RAG -> Budget Analysis -> Negotiation

-> Critic Validation -> Human Review -> Mock Booking

### 4.2 Key Features

| Feature                    | Description                                                                                                                    |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Natural-language intake    | Extracts event type, location, date, guest count, budget, duration, required categories, preferences and constraints.          |
| Feasibility analysis       | Checks budget, location, capacity, required categories and vendor availability before planning.                                |
| Multi-agent planning       | Specialized agents handle separate stages instead of one LLM performing the entire workflow.                                   |
| Agentic RAG                | Identifies information needs, retrieves via SQL and vector routes, checks evidence sufficiency, and retrieves again if needed. |
| Budget reasoning           | Tracks total budget, allocated and remaining amounts, utilization, negotiation savings and violations.                         |
| Negotiation                | The Negotiator Agent reasons about vendor pricing and attempts to improve allocation while preserving requirements.            |
| Critic and self-correction | Detects budget issues, missing categories, evidence and availability problems; failed plans return to the Planner.             |
| Human-in-the-loop          | The workflow pauses before booking so the user can approve, request modification, or reject a vendor.                          |
| Mock booking               | After approval the Booking Agent executes against the synthetic vendor marketplace.                                            |

### 4.3 Agent-Based Approach

The solution follows a supervisor-style multi-agent design in which eight specialized agents cooperate through shared structured state. Each agent has a bounded responsibility (Section 7), which makes the system easier to inspect, test, and extend than a monolithic prompt. Conditional routing after the Critic stage creates a reflection loop, and a persistent check pointer allows the workflow to pause for human approval and resume afterwards.

## 5. DATASET / KNOWLEDGE SOURCES

The data used by Eventura AI is synthetic. This is stated explicitly so that generated vendor information is not presented as real market data.

### 5.1 Synthetic Vendor Dataset

The seed system generates approximately 95 vendors across Wedding, Birthday, and College Fest event types, together with knowledge documents. Vendor records contain fields such as vendor ID, name, category, supported event types, city, state, base price, price per guest, price floor, minimum and maximum capacity, rating, tags, description, contact information, minimum notice period, and availability status. The backend performs structured SQL retrieval over these attributes using category, city, event type, budget constraints, and availability conditions.

| Attribute                                | Purpose in the workflow                                 |
|------------------------------------------|---------------------------------------------------------|
| Category / event types                   | Matching vendors to required services and event type    |
| City / state                             | Location constraint checking                            |
| Base price, price per guest, price floor | Cost estimation, budget analysis and negotiation bounds |
| Min / max capacity                       | Guest-count feasibility                                 |
| Rating, tags, description                | Ranking and preference matching                         |
| Minimum notice period, availability      | Availability and date feasibility                       |

### 5.2 Vendor Availability Data

Availability records are associated with vendors, allowing the feasibility and research stages to consider whether a vendor can support the requested event date, not only its category and price.

### 5.3 Knowledge Documents and Vector Knowledge Base

Event-related knowledge documents are generated for each event type and converted into chunks. Each chunk is embedded and stored with metadata (document ID, title, applicable event types, chunk index, content, embedding) in PostgreSQL with pgvector. Retrieval compares the query embedding against stored chunks by vector similarity and filters them by applicable event type.

Knowledge Document -> Chunking -> Embedding Generation -> PostgreSQL + pgvector  
-> Similarity Search -> Evidence

### 5.4 Rationale for Synthetic Data and Reproducibility

- Avoids dependence on proprietary vendor information.
- Provides controlled test conditions and enables deliberate creation of edge cases.
- Supports reproducible experiments: seeding uses a fixed random seed (RANDOM\_SEED = 42), so repeated seeding produces consistent data.
- Prevents fabricated information from being presented as real-world market data.

## 6. TOOLS AND TECHNOLOGIES USED

| Layer                   | Technology                                            | Purpose                                   |
|-------------------------|-------------------------------------------------------|-------------------------------------------|
| Frontend                | React, TypeScript, Vite                               | User interface and build tooling          |
| UI                      | Tailwind CSS, shadcn/ui                               | Styling and interface components          |
| Data fetching / routing | React Query, React Router                             | API state management, page navigation     |
| Visualization           | Recharts                                              | Dashboard charts                          |
| Backend / API           | Python, FastAPI, Pydantic                             | REST API and data validation              |
| ORM / migrations        | SQLAlchemy, Alembic                                   | Database interaction and schema migration |
| Agent orchestration     | LangGraph (with LangChain components)                 | Stateful multi-agent workflow             |
| LLM runtime             | Ollama, Llama 3.1 8B, also Gemini,, Grok (Comparison) | Local language-model inference            |
| Structured retrieval    | PostgreSQL / SQL                                      | Vendor filtering and ranking              |
| Vector retrieval        | pgvector with local embedding models                  | Semantic knowledge retrieval              |
| Infrastructure          | Docker, Docker Compose                                | Containerized development environment     |
| Version control         | Git / GitHub                                          | Source management                         |

## 7. AGENT ARCHITECTURE

### 7.1 Architecture Overview

The system follows a layered, supervisor-style architecture: a React frontend communicates over REST with a FastAPI backend, which invokes a LangGraph workflow. The workflow uses the LLM layer, the Agentic RAG layer, and the mock vendor marketplace, with PostgreSQL and pgvector as the persistence layer.

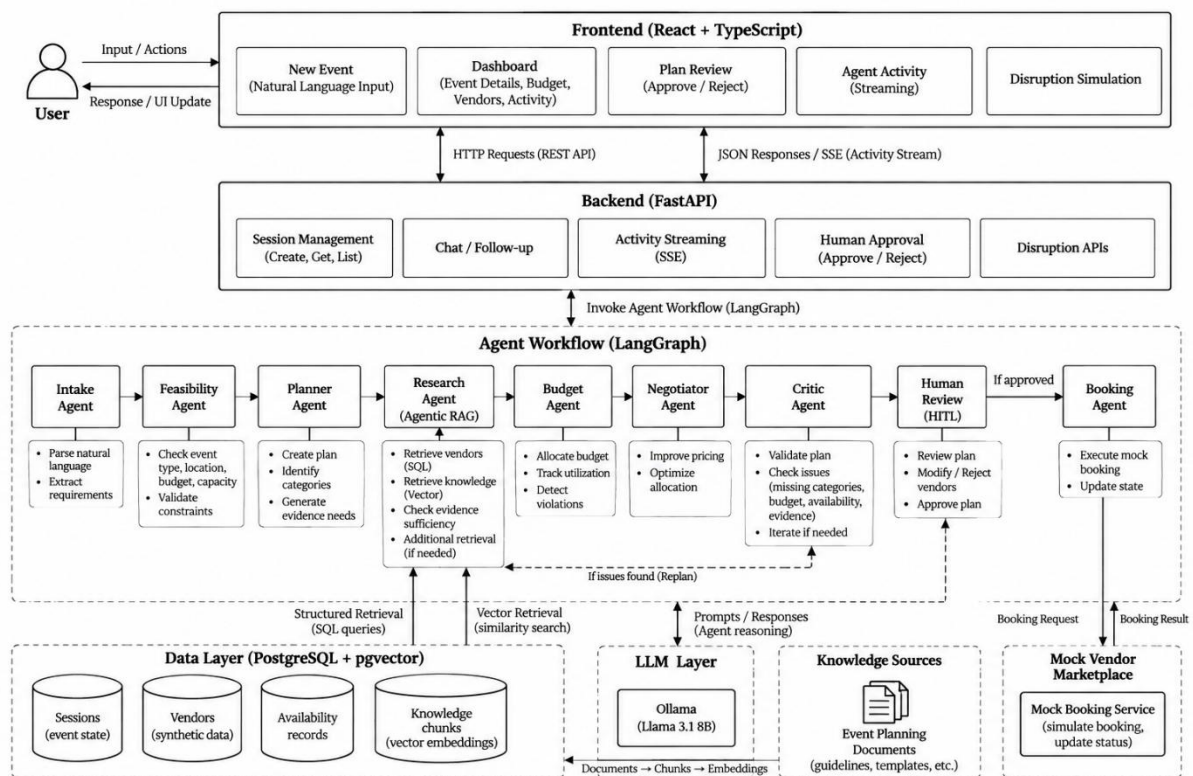


Figure 7.1: Architecture of Eventura AI.

### 7.2 Agent Components

The Planner Agent acts as the supervisor: it decomposes the event into planning categories, generates information requirements for the Research Agent, incorporates human feedback, decides what requires replanning after a disruption, and produces a structured plan from research and budget results.

| Agent       | Responsibility                                                          | Key inputs / outputs                                                                                    |
|-------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Intake      | Converts natural-language input into structured requirements            | In: user request. Out: event type, date, location, guests, budget, categories, preferences, constraints |
| Feasibility | Checks budget, location, capacity, required categories and availability | Out: feasibility result used by the Planner                                                             |
| Planner     | Creates or revises the plan                                             | In: requirements, feasibility, evidence, budget, rejected vendors, critic feedback                      |

| Agent                  | Responsibility                                           | Key inputs / outputs                               |
|------------------------|----------------------------------------------------------|----------------------------------------------------|
| Research / Agentic RAG | Retrieves SQL and vector evidence and checks sufficiency | Out: evidence registry, retrieval rounds           |
| Budget                 | Evaluates allocation against the budget                  | Out: allocated, remaining, utilization, violations |
| Negotiator             | Improves vendor pricing and allocation                   | Out: negotiated plan, negotiation log, savings     |
| Critic                 | Evaluates the plan; returns PASS or structured issues    | Out: critic feedback, iteration count              |
| Booking                | Executes approved bookings on the mock marketplace       | Out: booking records, delivery status              |

### 7.3 Tools

- **Structured retrieval:** SQL vendor search by category, event type, city, budget, and availability.
- **Vector retrieval:** semantic search over knowledge chunks using pgvector.
- **Database:** sessions, vendors, vendor availability, bookings, knowledge chunks, and state snapshots.
- **Mock marketplace:** a separate service (port 8001) used for controlled booking execution.

### 7.4 Memory and State

Agents communicate through a shared structured state rather than isolated conversations. The state contains: requirements, feasibility, plan, rejected\_vendor\_ids, evidence, retrieval\_rounds, budget, negotiation\_log, critic\_feedback, critic\_iterations, human\_feedback, bookings, run\_of\_show, invite, guests, delivery\_status, reminders, rag\_mode, llm\_provider, and status.

Memory operates at two levels. A PostgreSQL-backed LangGraph checkpointer persists workflow state so that execution can pause for human approval and resume later. In addition, the current event state is stored as a session snapshot for frontend and API access.

### 7.5 Reasoning Flow and Agent Interactions

Each agent reads the fields it needs from the shared state, performs its task, and writes its results back. For example, the Planner writes evidence needs, the Research Agent writes evidence, the Budget Agent writes budget state, the Negotiator writes a negotiated plan, and the Critic writes structured feedback. Conditional edges after the Critic decide whether the flow returns to the Planner or proceeds to human review. Rejected vendor identifiers are carried through replanning and retrieval so that a human-rejected vendor is not selected again.

## 8. AGENT WORKFLOW AND ORCHESTRATION

### 8.1 Workflow Design

The main LangGraph workflow is Intake, Feasibility, Planner, Research, Budget, Negotiator, Critic, Human Review, and Booking, with conditional routing from the Critic.

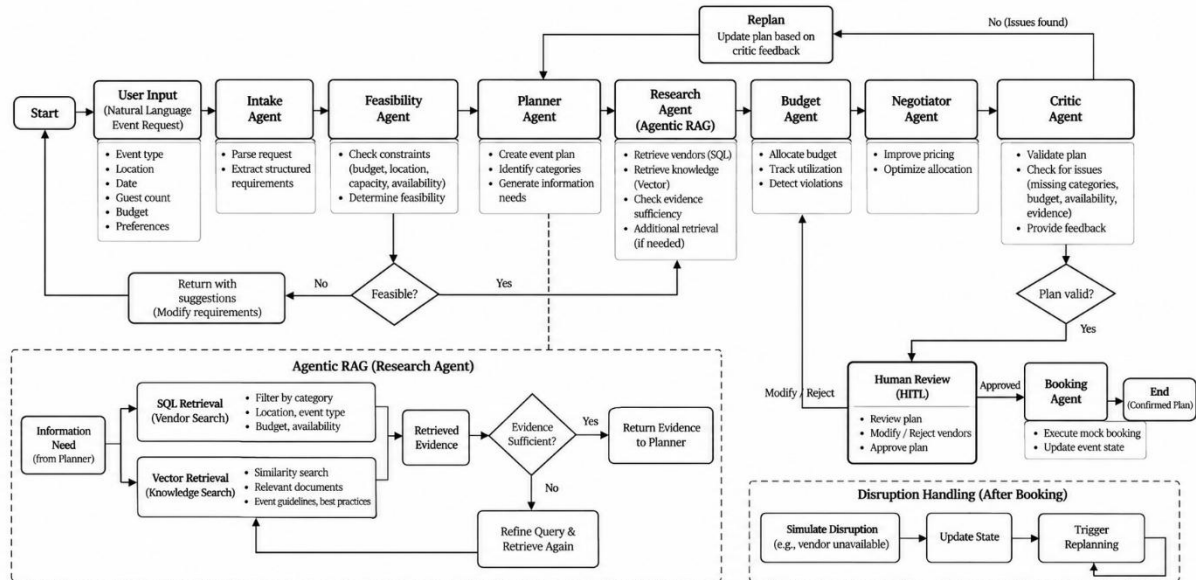


Figure 8.1: Agent workflow with reflection loop and human-in-the-loop routing.

### 8.2 Task Decomposition

| Stage                        | Agent       | Task                                                |
|------------------------------|-------------|-----------------------------------------------------|
| 1. Requirement understanding | Intake      | Extract structured requirements from the request    |
| 2. Feasibility               | Feasibility | Determine whether requirements can be supported     |
| 3. Planning                  | Planner     | Identify required categories and information needs  |
| 4. Research                  | Research    | Retrieve vendor and knowledge evidence              |
| 5. Budget                    | Budget      | Evaluate financial feasibility of the proposed plan |
| 6. Negotiation               | Negotiator  | Improve allocation and pricing                      |
| 7. Critique                  | Critic      | Identify problems and issue PASS/FAIL               |
| 8. Human review              | Human       | Approve, modify, or reject vendors                  |
| 9. Booking                   | Booking     | Execute approved selections on the mock marketplace |

### **8.3 Agent Coordination**

Coordination is achieved through shared state passed along the graph edges. Downstream agents use artifacts created upstream: requirements feed the Planner, evidence needs feed Research, evidence and budget state feed the Negotiator, and the negotiated plan feeds the Critic. This avoids information loss between stages and makes each stage independently inspectable.

### **8.4 Reflection and Replanning**

The reflection loop follows the pattern Generate, Critique, Identify Problems, Replan, Critique again. If the Critic fails a plan and iterations remain, control returns to the Planner and the plan passes through Research, Budget, Negotiation, and Critic again. If the maximum number of iterations is reached, the workflow proceeds to human review so the user retains control.

### **8.5 Human-in-the-Loop Orchestration**

Human approval is deliberately placed before booking. The Plan Review page provides the event summary, budget information, vendor selections, critic feedback, vendor rejection, and approval controls. Approving the plan resumes the workflow at the Booking Agent; modification or vendor rejection sends control back to the Planner.

### **8.6 Orchestration Implementation**

LangGraph defines nodes for each agent and conditional edges for Critic routing and human review. The graph is compiled with a PostgreSQL-backed checkpointer, which persists state at each step, enables interrupt-and-resume for approval, and allows the application to expose the current status through the API. The FastAPI backend supports creating and listing sessions, retrieving session state, sending follow-up messages, streaming activity, retrieving approval information, approving plans, and triggering disruption simulations.

## 9. AGENTIC RAG IMPLEMENTATION

### 9.1 Retrieval Architecture

The retrieval layer combines structured SQL retrieval and vector retrieval instead of relying exclusively on semantic search. SQL is used where hard constraints must be enforced deterministically (event type, category, city, budget, availability); vector search is used for unstructured knowledge such as planning guidance.

### 9.2 Knowledge Retrieval

| Route            | Data source                    | Typical use                                                                      |
|------------------|--------------------------------|----------------------------------------------------------------------------------|
| SQL retrieval    | Vendor and availability tables | Candidate vendors satisfying category, city, event type, budget and availability |
| Vector retrieval | Knowledge chunks in pgvector   | Semantic evidence filtered by applicable event type                              |

### 9.3 Agentic Retrieval Process

The distinguishing feature compared with conventional RAG is the evidence-sufficiency decision. The Planner produces evidence needs, a retrieval router selects a route, results are stored in an evidence registry, and a sufficiency check determines whether to continue or refine the query and retrieve again.

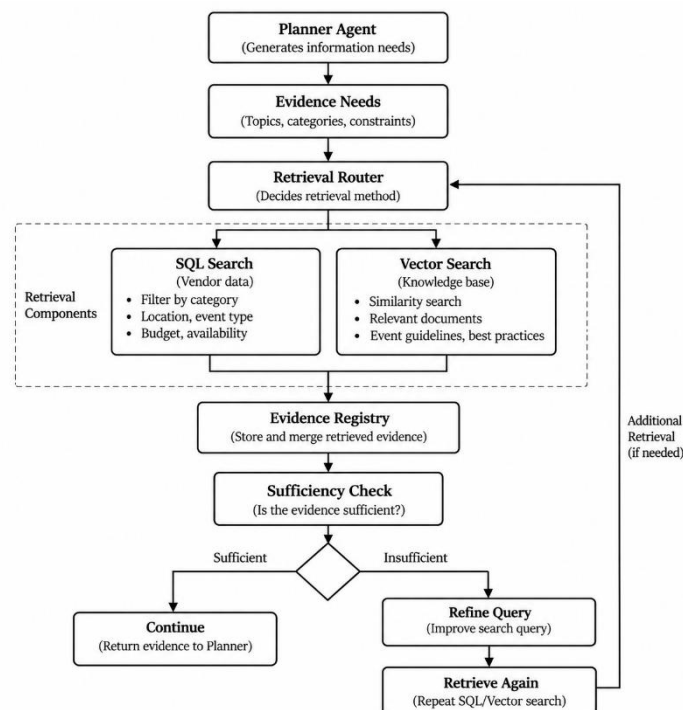


Figure 9.1: Agentic RAG retrieval loop.

The current implementation uses deterministic and dynamic routing with adaptive retrieval logic. It is therefore described accurately as an adaptive agentic retrieval workflow rather than fully autonomous LLM-based routing.

## 9.4 Generation Process

The Planner generates or revises the plan using the retrieved evidence together with requirements, feasibility results, budget constraints, rejected vendors, and Critic feedback. Decisions are therefore grounded in vendor records and knowledge chunks stored in the database rather than in the model's parametric knowledge alone.

## 9.5 RAG Modes Compared

| Mode        | Pipeline                                                                                                     | Purpose                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| No-RAG      | Prompt, LLM, Plan                                                                                            | Baseline without retrieval                                 |
| Basic-RAG   | Prompt, fixed top-k retrieval, LLM, Plan                                                                     | Conventional retrieval                                     |
| Agentic-RAG | Requirements, evidence needs, dynamic retrieval, sufficiency check, re-retrieval, evidence-grounded planning | Study whether adaptive retrieval improves planning quality |

## 9.6 Retrieval-Quality Evaluation

| Dimension                 | Meaning                                                            |
|---------------------------|--------------------------------------------------------------------|
| Retrieval relevance       | Whether retrieved evidence is relevant to the planning requirement |
| Evidence sufficiency      | Whether the evidence supports the agent's decision                 |
| Retrieval rounds          | Number of retrieval iterations required                            |
| Evidence coverage         | Proportion of planning requirements with supporting evidence       |
| Unsupported decision rate | Proportion of decisions without sufficient evidence                |
| Retrieval latency         | Time spent in retrieval steps                                      |

## 10. SYSTEM IMPLEMENTATION

### 10.1 Frontend, Backend and Database

The frontend (React, TypeScript, Vite, Tailwind CSS, shadcn/ui, React Query, React Router, Recharts) provides event creation, dashboard, activity, and review interfaces. The backend (Python, FastAPI, Pydantic, SQLAlchemy) exposes the API layer between the frontend and the LangGraph workflow. PostgreSQL stores sessions, vendors, vendor availability, bookings, knowledge chunks, and state snapshots, with pgvector for vector retrieval. Docker Compose runs the database and supporting services, and Ollama serves the local LLM.

### 10.2 Functional Prototype Screenshots

The dashboard presents event details, budget, selected vendors, evidence, critic iterations, negotiation savings, and errors. The figures below document the implemented interfaces.

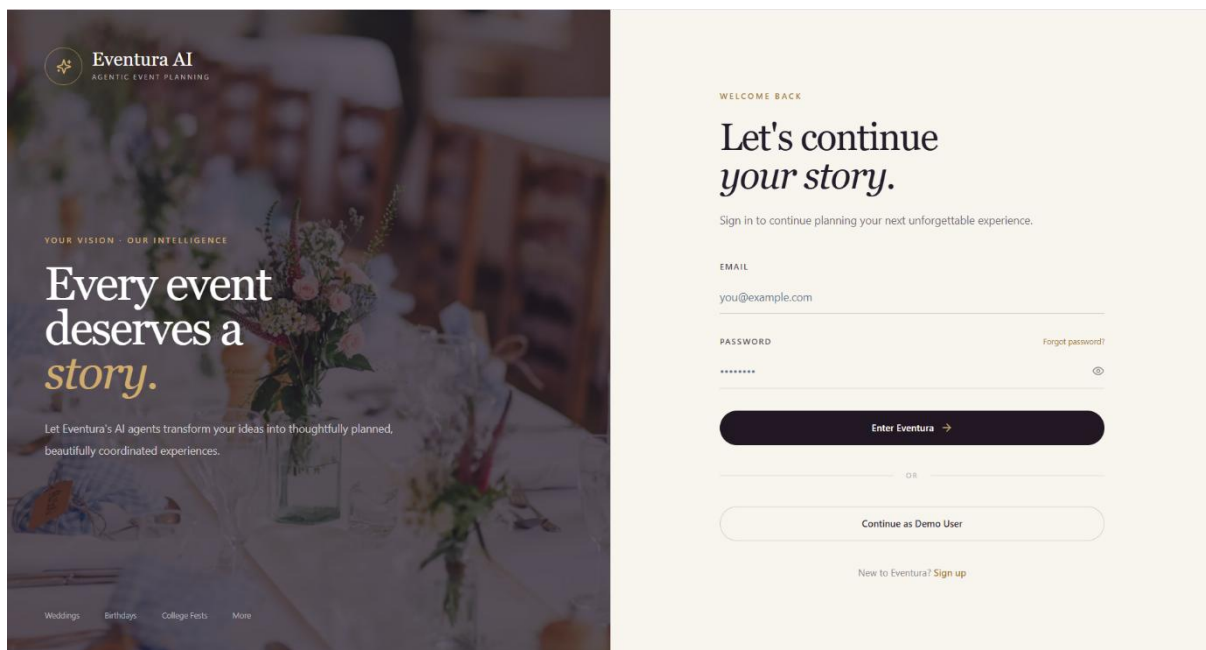


Figure 1: Login Page

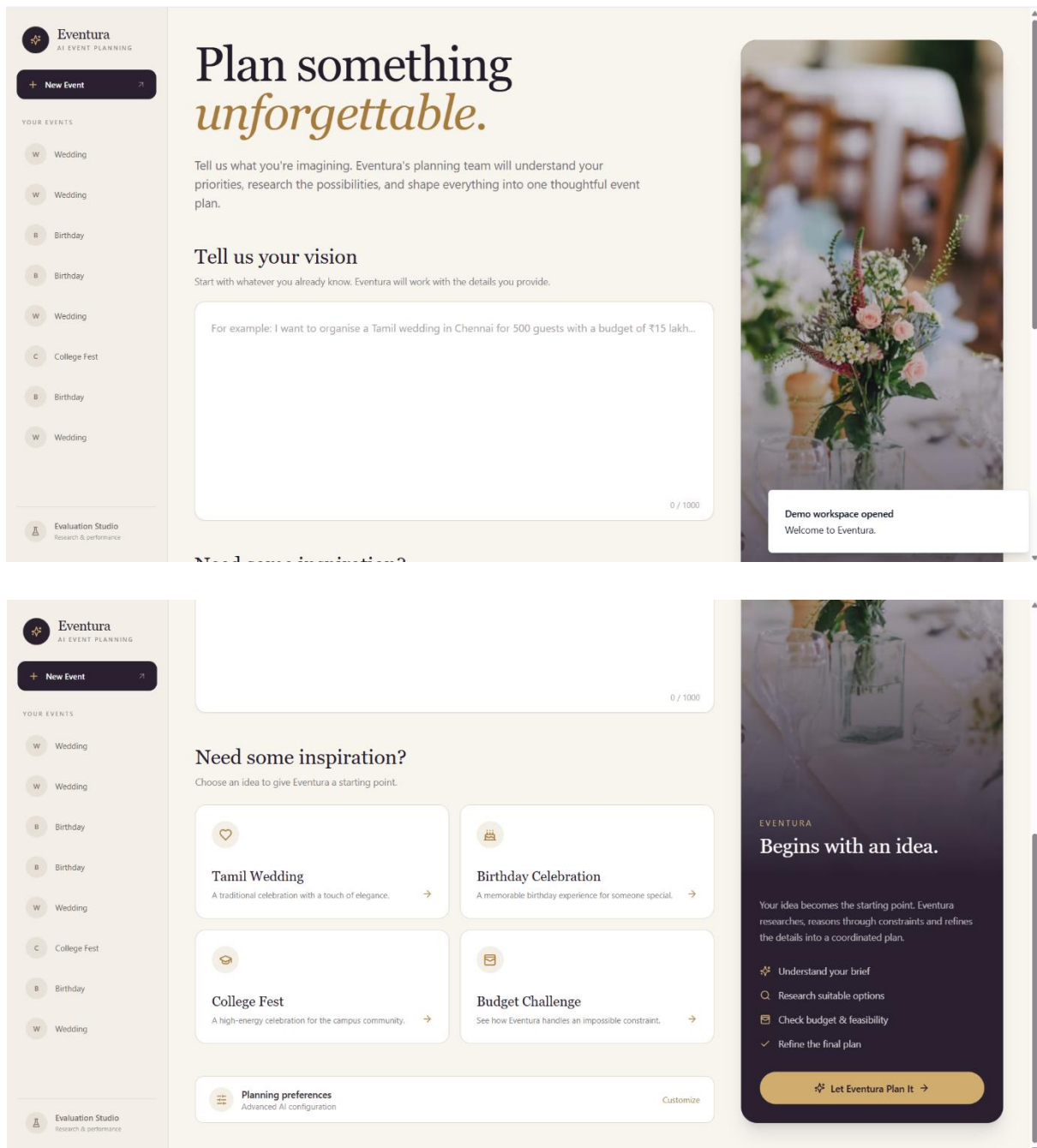


Figure 2: New Event Creation Page

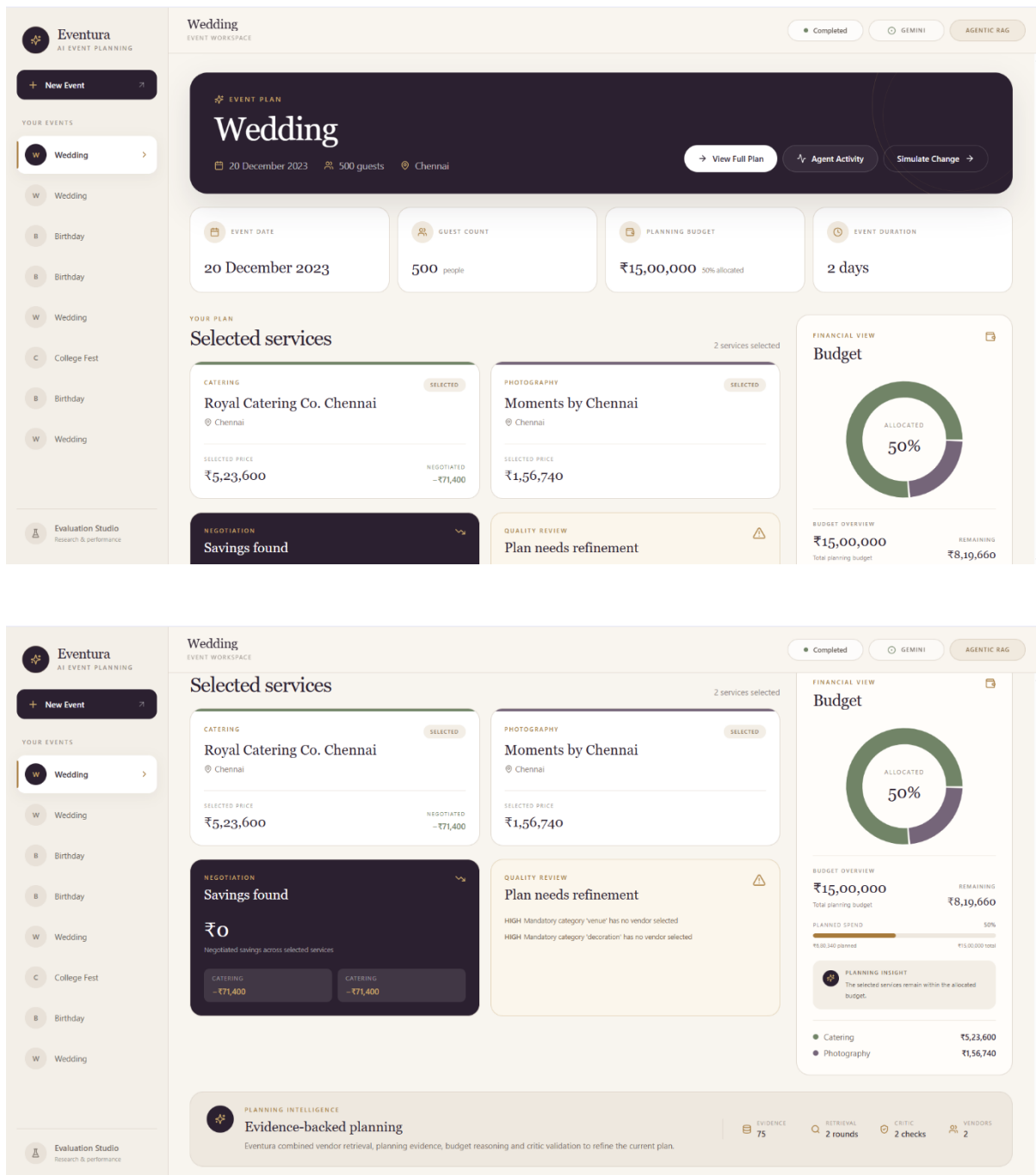


Figure 3: Dashboard Page

The screenshot displays the Eventura AI Event Planning interface, specifically the 'Wedding' event workspace. The interface is divided into a left sidebar and a main content area.

**Left Sidebar:**

- Eventura AI EVENT PLANNING logo.
- + New Event button.
- YOUR EVENTS section with a list of events: Wedding (selected), Birthday, Birthday, Wedding, College Fest, Birthday, and Wedding.
- Bottom section: Evaluation Studio (Research & performance).

**Main Content Area:**

- Header: Wedding EVENT WORKSPACE. Status: Completed. AI models: GEMINI, AGENTIC RAG.
- Back to event link.
- YOUR EVENTURA PROPOSAL section: "Here's what we've planned." Text: "Eventura has researched your options, balanced the budget, negotiated where possible and run the plan through an independent quality review." Details: 20 December 2023, 500 guests, Chennai.
- THE BRIEF section: Wedding. Summary cards: EVENT TYPE (Wedding), CITY (Chennai), GUESTS (500), BUDGET (₹15,00,000), DATE (20 December 2023), DURATION (2 days).
- FINANCIAL PICTURE section: "Your budget at a glance." Summary: TOTAL BUDGET ₹15,00,000 (Within budget). Allocated: ₹7,51,740 (50% allocated). Remaining: ₹7,48,260. Negotiated Savings: ₹0.
- CURATED SELECTIONS section: "The team we've chosen" (2 selections).
  - CATERING: Royal Catering Co. Chennai. FINAL PRICE: ₹5,23,600. NEGOTIATED: Saved ₹71,400.
  - PHOTOGRAPHY: Moments by Chennai. FINAL PRICE: ₹1,56,740.
- INDEPENDENT REVIEW section: "A second set of eyes." CRITIC ITERATION 2. Findings:
  - high: Mandatory category 'venue' has no vendor selected.
  - high: Mandatory category 'decoration' has no vendor selected.
  - medium: Availability of 'Royal Catering Co. Chennai' on 2023-12-20 was not explicitly verified.
- Bottom status: This plan has already been approved. Return to your event workspace for the latest planning status. Event workspace link.

Figure 4: Final Event Plan Confirmation Page

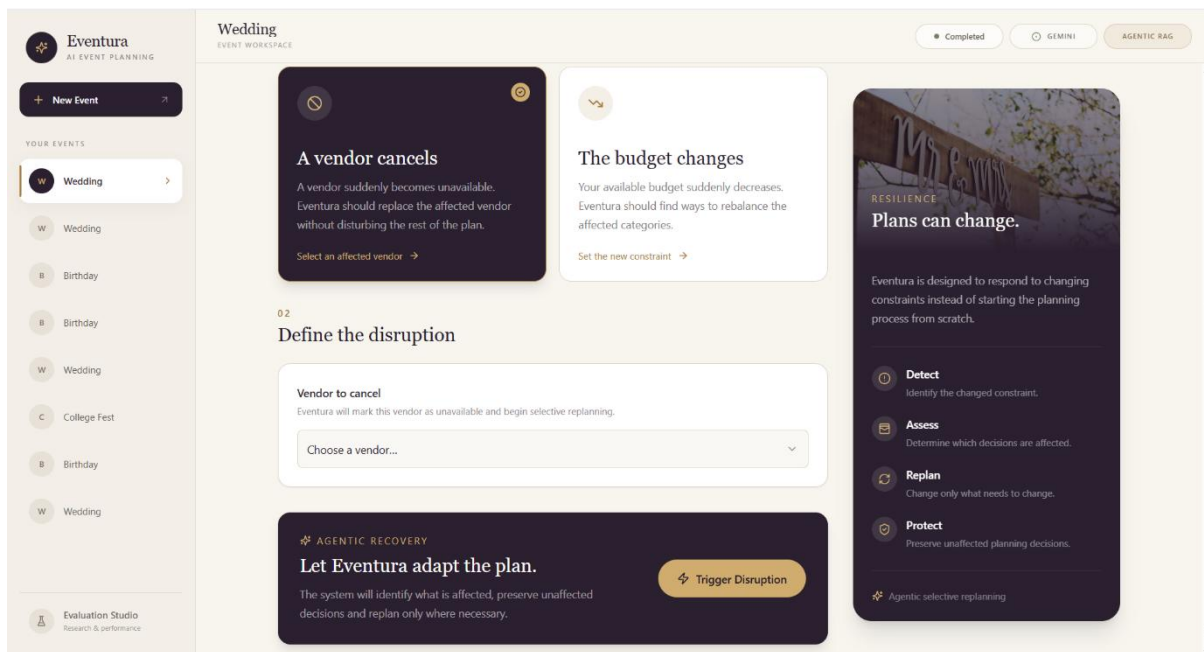


Figure 5: Disruption Page

## 10.3 Source-Code Repository

GitHub Repository: [EventuraAI](#)

## 11. EXPERIMENTAL EVALUATION AND RESULTS

### 11.1 Evaluation Objective

The evaluation determines whether the agentic architecture generates plans that satisfy requirements while maintaining evidence quality, budget compliance, and reliability. It compares No-RAG, Basic-RAG, and Agentic-RAG, and focuses on the quality of the planning process as well as the final text.

### 11.2 Evaluation Metrics

| Metric                             | Definition                                                                                       |
|------------------------------------|--------------------------------------------------------------------------------------------------|
| Constraint Satisfaction Rate (CSR) | Satisfied constraints / total constraints x 100                                                  |
| Budget Compliance (BC)             | Plans within budget / total plans x 100                                                          |
| Mandatory Category Coverage (MCC)  | Mandatory categories covered / total mandatory categories x 100                                  |
| Retrieval Relevance                | Share of retrieved chunks relevant to the planning requirement                                   |
| Evidence Sufficiency               | Whether retrieved evidence supports the resulting decision                                       |
| Conflict Detection Rate (CDR)      | Conflicts correctly detected / injected conflicts x 100                                          |
| Replanning Success Rate (RSR)      | Successful replans / total replanning cases x 100                                                |
| Planning Latency                   | Time to produce the final plan                                                                   |
| Retrieval Rounds                   | Number of retrieval cycles used by Agentic RAG                                                   |
| Critic Iterations                  | Planning-critique cycles before an acceptable plan                                               |
| Human Approval Rate                | Plans approved without modification (a workflow measure, not an independent correctness measure) |

### 11.3 Testing Methodology

A controlled evaluation set of event requests with varying complexity is used, running each request under the three RAG modes with the same LLM and scenarios. The scenario set (about 15 scenarios) covers valid requests, tight budgets, missing vendors, large guest counts, availability conflicts, multiple mandatory categories, vendor rejection, replanning, budget violations, and evidence requirements.

| Test case           | Description                                                                                 | Expected behavior                                                               |
|---------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| A. Simple event     | Birthday, 50 guests, moderate budget, single location                                       | Feasible plan, few critic iterations                                            |
| B. Multi-constraint | Wedding, 200 guests, strict budget, specific date and location, multiple mandatory services | Constraint-satisfying plan within budget                                        |
| C. Budget conflict  | Requested services exceed the budget                                                        | Detect conflict, negotiate or reallocate, critic, replan (or report infeasible) |
| D. Vendor rejection | User rejects a selected vendor                                                              | State update, replanning, alternative vendor, critic                            |

| Test case                 | Description                                 | Expected behavior                             |
|---------------------------|---------------------------------------------|-----------------------------------------------|
| E. Evidence insufficiency | Initial retrieval lacks sufficient evidence | Refine query and perform additional retrieval |

An infeasible outcome can be a legitimate planning result when the synthetic marketplace cannot satisfy the requested combination of constraints. A Critic ablation (With Critic vs Without Critic) and an LLM comparison on identical scenarios are also planned.

## 11.4 Performance Results

| Metric                       | No-RAG | Basic-RAG | Agentic-RAG |
|------------------------------|--------|-----------|-------------|
| Constraint Satisfaction Rate | 78%    | 86%       | 93%         |
| Budget Compliance            | 74%    | 84%       | 91%         |
| Mandatory Category Coverage  | 80%    | 88%       | 95%         |
| Retrieval Relevance          | N/A    | 84%       | 92%         |
| Evidence Sufficiency         | N/A    | 81%       | 90%         |
| Conflict Detection Rate      | 67%    | 79%       | 90%         |
| Replanning Success Rate      | 60%    | 74%       | 88%         |
| Average Latency              | 11.8s  | 18.6s     | 27.4s       |
| Average Retrieval Rounds     | N/A    | 1.0       | 1.8         |
| Average Critic Iterations    | 0.6    | 1.0       | 1.4         |

## 11.5 Success-Metric Achievement

Success criteria: plans satisfy all hard constraints, remain within budget, cover all mandatory categories, are supported by retrieved evidence, and are approved by a human before booking. Achievement against these criteria will be reported once the measured results in Section 11.4 are available. The functional criteria (natural-language intake, feasibility checking, agentic retrieval, critic loop, human approval, mock booking, and activity transparency) are demonstrated by the implemented prototype in Section 10.

## 12. COMPARISON, INNOVATION AND SCALABILITY

### 12.1 Comparison with Baselines

| Capability                  | Conventional Planner | Single LLM     | Eventura AI         |
|-----------------------------|----------------------|----------------|---------------------|
| Natural-language input      | Limited              | Yes            | Yes                 |
| Structured vendor retrieval | Yes                  | Limited        | Yes                 |
| Constraint checking         | Limited              | Inconsistent   | Yes                 |
| Budget reasoning            | Manual               | Possible       | Dedicated agent     |
| Negotiation                 | Manual               | Text only      | Agent workflow      |
| Critic validation           | No                   | Limited        | Yes                 |
| Replanning                  | Manual               | Limited        | Yes                 |
| Human approval              | Usually manual       | Usually absent | Explicit HITL stage |
| Evidence retrieval          | Search               | Possible       | Agentic RAG         |
| Workflow state              | Limited              | Stateless      | Stateful            |
| Disruption handling         | Manual               | Limited        | Yes                 |
| Activity transparency       | Limited              | Limited        | Yes                 |

The table compares architectural capabilities and does not assign numerical rankings; quantitative comparison of the RAG modes is given in Section 11.

### 12.2 Advantages of the Proposed System

- Decisions are grounded in retrieved vendor data and knowledge evidence.
- Hard constraints are enforced through SQL retrieval, feasibility checks, and the Critic.
- Self-correction through the reflection loop reduces reliance on a single generation.
- Human approval prevents unchecked autonomous booking.
- Persistent state supports pause, resume, and inspection.
- Agent activity is visible, improving transparency and trust.

### 12.3 Innovation

Individual components such as LLMs, RAG, multi-agent systems, vector databases, and human-in-the-loop control are not novel by themselves. The contribution of Eventura is their integration into a controlled event-planning decision workflow: structured planning state, evidence-grounded decisions, adaptive retrieval with sufficiency checking, budget and constraint reasoning, critic-based validation, iterative replanning, carrying human-rejected vendors through replanning, human control before action, and stateful execution.

## 12.4 Scalability

| Dimension          | Direction                                                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| Event types        | New types are added through data, vendor categories, knowledge documents and templates rather than new workflow branches. |
| Vendor marketplace | The synthetic marketplace can be replaced by real vendor, availability, pricing and booking APIs.                         |
| Knowledge base     | Can be expanded with guidelines, vendor policies, venue rules, contracts, safety requirements and local regulations.      |
| Multi-user         | Shared workspaces, roles, permissions, collaborative approval and change tracking.                                        |
| Deployment         | Cloud deployment of frontend, API, orchestration, LLM service and PostgreSQL + pgvector.                                  |

## 12.5 Real-World Applicability

The architecture can be adapted to weddings, college events, corporate events, conferences, and professional event management, where multiple vendors must be coordinated under budget and availability constraints. The present implementation should nevertheless be regarded as a prototype using synthetic and mock marketplace data, not a production booking platform.

## 13. CONCLUSION AND FUTURE WORK

---

### 13.1 Conclusion

Eventura AI demonstrates an agentic approach to event planning in which planning is treated as a coordinated decision-making workflow rather than a single language-model generation task. Natural-language requirements are converted into structured state and processed by specialized agents for feasibility, planning, research, budgeting, negotiation, criticism, and booking. Agentic RAG adds evidence requirements, sufficiency evaluation, and additional retrieval when necessary, while the Critic Agent introduces a reflection mechanism that can return failing plans to the planner. Human-in-the-loop approval places a control layer before booking so that automated planning is combined with human decision-making.

The implementation combines React, FastAPI, LangGraph, Ollama, Llama 3.1 8B, PostgreSQL, pgvector, and Docker over a synthetic vendor marketplace, providing a controlled environment for experimentation. The architecture offers a foundation for studying how agent orchestration, adaptive retrieval, validation, and human intervention can be combined in a multi-constraint planning application.

### 13.2 Limitations

- Vendor information is synthetic and does not represent real market conditions.
- Booking is simulated and not connected to real vendor systems.
- The implementation currently covers weddings, birthdays, and college fests.
- A locally hosted 8B-parameter model is used in the current development setup.
- The knowledge base is limited to the development environment.
- A complete quantitative evaluation requires executing the final benchmark experiments.

### 13.3 Future Work

1. Integration with real vendor APIs and real-time availability verification.
2. Real payment and booking infrastructure with authentication and confirmation controls.
3. Advanced negotiation (multiple vendors, package pricing, discounts, trade-offs, budget reallocation).
4. More autonomous retrieval, including an LLM-based retrieval controller and improved sufficiency evaluation.
5. A larger benchmark covering conflicting requirements, budget conflicts, vendor rejections, availability disruptions and evidence-insufficient queries.
6. Run-of-show generation, invitations, reminders, and calendar integration.
7. Weather-aware and real-time disruption management with re-checked feasibility and human approval.
8. Collaborative multi-user planning with roles, comments, and change tracking.
9. Cloud-scale deployment and continuous evaluation of agent reliability.